Category 1

```
MY SETUP:
WINDOWS 11
VSCODE
PLATFORMIO
WSL FOR ROS2 HUMBLE

Read and process data from onboard sensors (GPS, IMU, Encoder) on a Teensy 4.1
microcontroller, and optionally publish the data as ROS 2 topics using MicroROS.

**Objective:** Read GPS data from the 7Semi L86-M33 module and parse NMEA sentences.

**What You Will Implement:**
- Configure the appropriate baud rate
- Parse incoming NMEA sentences
- Extract: Latitude, Longitude, Altitude, Timestamp
- Print formatted GPS data to the serial monitor

**Expected Output:**
```
Lat: 13.195987
Lon: 80.224016
Altitude: 15.34 m
```
```

hi
so this is my first task

So first I started off with asking ChatGPT what am I getting into.
I already had Platform.io vscode extension so I think im good.

I took Platform IO, made a new project named teensy__GPS with the board as Teensy 4.1 .

Had some folder location issue. i was using the wrong workspace for platform io so i opened in the correct workspace.

So I have extracted the gitub repository of TinyGPS++ into /lib and I hope the import issue fixes.
Let me just add it to the lib_deps also for safetey.
now i compiled to check if the code is correct.

```
 *  Terminal will be reused by tasks, press any key to close it.

 *  Executing task: C:\Users\satya\.platformio\penv\Scripts\platformio.exe run
Checking size .pio\build\teensy41\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
Calculating size .pio\build\teensy41\firmware.elf
teensy_size: Memory Usage on Teensy 4.1:
teensy_size:   FLASH: code:16040, data:4040, headers:8588   free for files:8097796
teensy_size:    RAM1: variables:5184, code:13520, padding:19248   free for local variables:486336
teensy_size:    RAM2: variables:12416  free for malloc/new:511872
Building .pio\build\teensy41\firmware.hex
============================================================================= [SUCCESS] Took 2.60
 *  Terminal will be reused by tasks, press any key to close it.
```

Yes the code is correct
Now let me understand it

CODE EXPLANATION

SO THIS WAS THE CODE I USED FIRST

```
#include <Arduino.h>
#include <TinyGPS++.h>
TinyGPSPlus gps;
void setup() {
Serial.begin(115200);
Serial1.begin(9600);
}
void loop() {
while (Serial1.available()) {
gps.encode(Serial1.read());
}
if (gps.location.isUpdated() && gps.location.isValid() && gps.altitude.isValid()) {
    Serial.print("Lat: ");
    Serial.println(gps.location.lat(), 6);
    Serial.print("Lon: ");
    Serial.println(gps.location.lng(), 6);
    Serial.print("Altitude: ");
    Serial.print(gps.altitude.meters(), 2);
    Serial.println(" m");
    Serial.println();
}
}
```

There are no pin defininitions

no one knows which pin can be where

so what then?

Use HardwareSerial to define the pins.

HardwareSerial defined RX and TX at 0 and 1 automatically. But if you want to change, then you can. (Explained below)

```
#include <Arduino.h>
#include <TinyGPS++.h>
TinyGPSPlus gps;
HardwareSerial &gpsUart = Serial1;
```

- TinyGPSPlus is a lightweight library that parses NMEA sentences (standard GPS output format).

- gpsUart is a reference to Serial1, the hardware UART typically used for external devices like GPS modules.

```
constexpr uint32_t USB_SERIAL_BAUD = 115200;
constexpr uint32_t GPS_BAUD = 9600;
```

- Most GPS modules default to **9600 baud**.
- **115200 baud** for USB serial allows faster debugging/output to your computer.

```
void setup() {
Serial.begin(USB_SERIAL_BAUD);
// Serial.begin(9600, SERIAL_8N1, 16, 17); // (Baud, Config, RX_Pin, TX_Pin)
gpsUart.begin(GPS_BAUD); // basically serial1 begin
}
```

Here comes the change. We are using something called HardwareSerial Library.

Reference: Arduino Serial: A Look at the Different Serial Libraries – Martyn Currey

Hardware serial is built in to the Arduino hardware which means it has dedicated pins, the ones labelled RX and TX pins (pins 0 and 1). All Arduinos have at least one hardware serial channel. Some have more.

HardwareSerial is the Arduino-core class that wraps a microcontroller UART peripheral and provides buffered serial I/O (methods like begin(), available(), read(), write(), print(), flush()).

On Teensy (PlatformIO framework-arduinoteensy) hardware UARTs appear as Serial1, Serial2, etc. . It uses interrupt-driven RX with an internal ring buffer (faster and lower CPU cost than SoftwareSerial)

HardwareSerial is part of the Arduino core used by PlatformIO for Teensy () and is available by default; you don't need to add it to platformio.ini.:

in the example code found in

https://7semi.com/l86-m33-gps-gnss-breakout-board/?srsltid=AfmBOorb7h6uKE5F2D6D4SoCVoVuQPeEgG30AyWD7evtRqXZIzTI44tq

they use software serial.

but why did i switch to hardware serial.

from what i searched around, SoftwareSerial Library, DO NOT USE. NewSoftSerial works much better.

software serial is based on emulation

SoftwareSerial should be used only as a last resort, because it has many limitations:

- Transmitting can interfere with other libraries which require interrupts. Use of slow baud rates may cause more interference.
- Transmitting can block or interfere with SoftwareSerial reception.
- Use of interrupts by other libraries can interfere with SoftwareSerial reception, especially if using higher baud rates.
- If multiple SoftwareSerial ports are created, only 1 can work at a time.
- Only certain pins can receive on Teensy LC, Teensy 2.0 and Teensy 1.0.
- Teensyduino 1.59 and earlier: Transmitting does not work on Teensy 4.x. This will be fixed in version 1.60.
- Teensyduino 1.59 and earlier: Receiving does not work on Teensy 3.x or 4.x. This will be fixed in version 1.60.
- The inverse_logic option is not supported on Teensy 3.x or 4.x.

Hardware Serial is universally superior to Software Serial

Teensyduino ▸ Libraries ▸ SoftwareSerial

SoftwareSerial Library

Download: SoftwareSerial is included with Teensyduino. Source Code on Github.

Use Hardware Serial If Possible

```
void loop() {
while (gpsUart.available() > 0) {
const char nmeaByte = static_cast<char>(gpsUart.read());
gps.encode(nmeaByte);
}
```

the while condition checks if it is available or not, the .avaliable function returns a value other than 0, its giving values

arduino stream read reruns int and we static_casting to savely convert

we can use regular one also but still this seems safer.

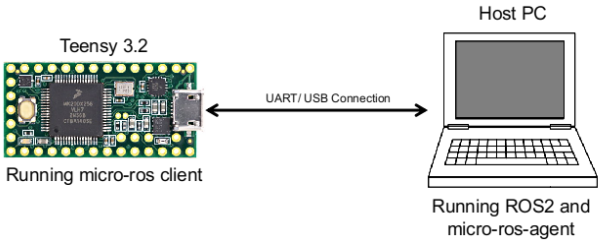
https://stackoverflow.com/questions/28002/regular-cast-vs-static-cast-vs-dynamic-cast

```
if (gps.location.isUpdated() && gps.location.isValid() && gps.altitude.isValid()) {
    Serial.print("Lat: ");
    Serial.println(gps.location.lat(), 6);

    Serial.print("Lon: ");
    Serial.println(gps.location.lng(), 6);

    Serial.print("Altitude: ");
    Serial.print(gps.altitude.meters(), 2);
    Serial.println(" m");

    Serial.println();
}
```

}

TEENSY AND MICROROS NOW

Teensy with Arduino | micro-ROS



view the image in light mode pls

I already have ROS2 Humble installed on WSL2 so i think im cool for now.

so im thinking like
Since MicroROS will be on the teensy
and I will Make MicroROS Agent in WSL and this will communicate with ROS2
~~So ill make one listener in ROS 2 which will take all that and print in terminal like a serial monitor~~ (I didn't do that)

Building MicroROS AGENT on WSL

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ ros2 topic list
/parameter_events
/rosout
```

right now there are no ROS nodes or topics being published

First micro-ROS Application on Linux | micro-ROS

I have now made a folder called microros_src and i did git clone

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1$ mkdir microros_src
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1$ cd microros_src/
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1/microros_src$ git clone -b humble https://github.com/micro-ROS/micro-ROS-Agent.git
Cloning into 'micro-ROS-Agent'...
remote: Enumerating objects: 1431, done.
remote: Counting objects: 100% (452/452), done.
remote: Compressing objects: 100% (131/131), done.
remote: Total 1431 (delta 410), reused 321 (delta 321), pack-reused 979 (from 3)
Receiving objects: 100% (1431/1431), 312.28 KiB | 1.43 MiB/s, done.
Resolving deltas: 100% (729/729), done.
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1/microros_src$
```

sorted.

now install ros dependencies and then build the agent

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1/microros_ws$ rosdep install --from-paths src --ignore-src -y
/usr/bin/rosdep:6: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import load_entry_point
#All required rosdeps installed successfully
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category 1/microros_ws$ colcon build
Starting >>> micro_ros_agent
[Processing: micro_ros_agent]
[Processing: micro_ros_agent]
[Processing: micro_ros_agent]
[Processing: micro_ros_agent]
[Processing: micro_ros_agent]
```

and now we source it

`source install/setup.bash`

Getting micro_ros_arduino on Teensy

micro-ROS/micro_ros_arduino: micro-ROS library for Arduino

https://micro.ros.org/docs/overview/rtos/#:~:text=micro%2DROS%20aims%20to%20bring,of%20the%20supported%2

this is a micro-ROS library for baremetal projects based on Arduino IDE or Arduino CLI. We are directly using micro_ros_arduino

Since i am using micro_ros_Arduino library, i do not need to worry about any RTOS and all. We are simply using

Bare metal support

Based on the release of micro-ROS as a standalone library with header files, and on the support provided to the A
IDE, micro-ROS is available as a bare-metal application, too. Find more details in the dedicated repo.

Arduino bare-metal support

The open-source Arduino Software (IDE) is a library making it easy to program any Arduino board.

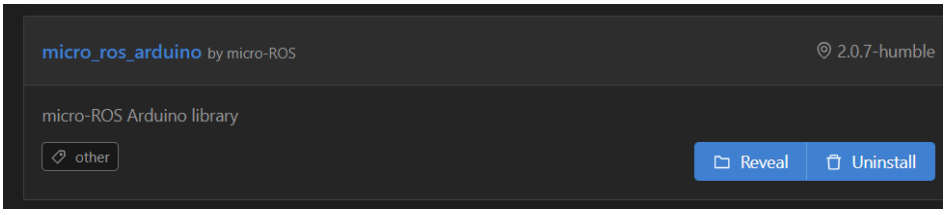
Key features:

- Inexpensive
- Cross-platform
- Simple, clear programming environment
- Open source and extensible software
- Open source and extensible hardware

Resources:

- Official Arduino Website
- micro_ros_arduino repo

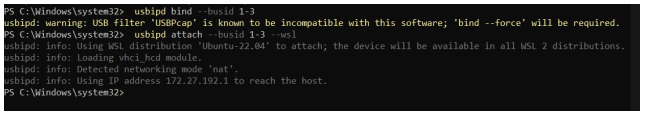
I have now installed micro_ros_arduino library on platform io



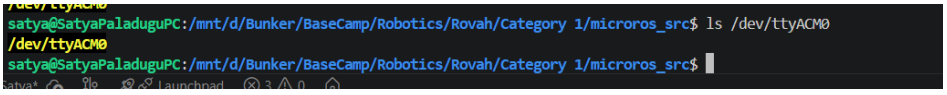
for safety, i have added to .ini file also

my code for teensy using `micro_ros_arduino.h` is at the bottom of the report

I will now bind and attach my usb port to the wsl



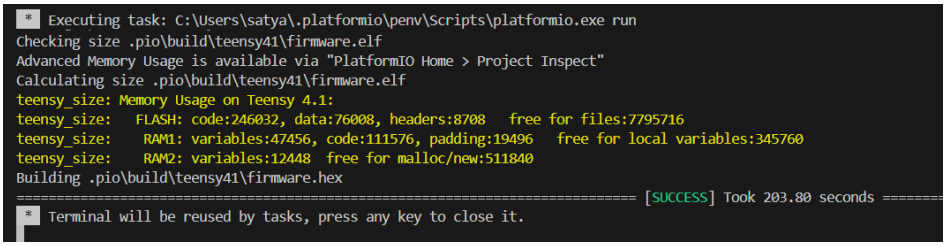
lemme check if my thing is coming.



Putting both of them together.

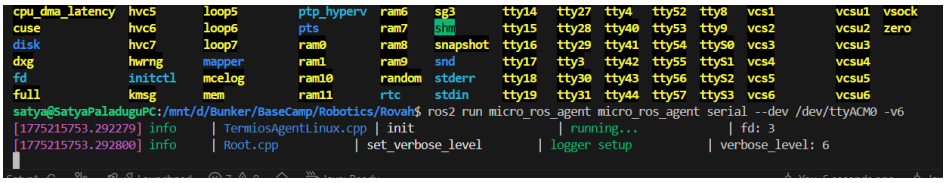
lemme build my platformio code

bro is ready



now let's get the ros

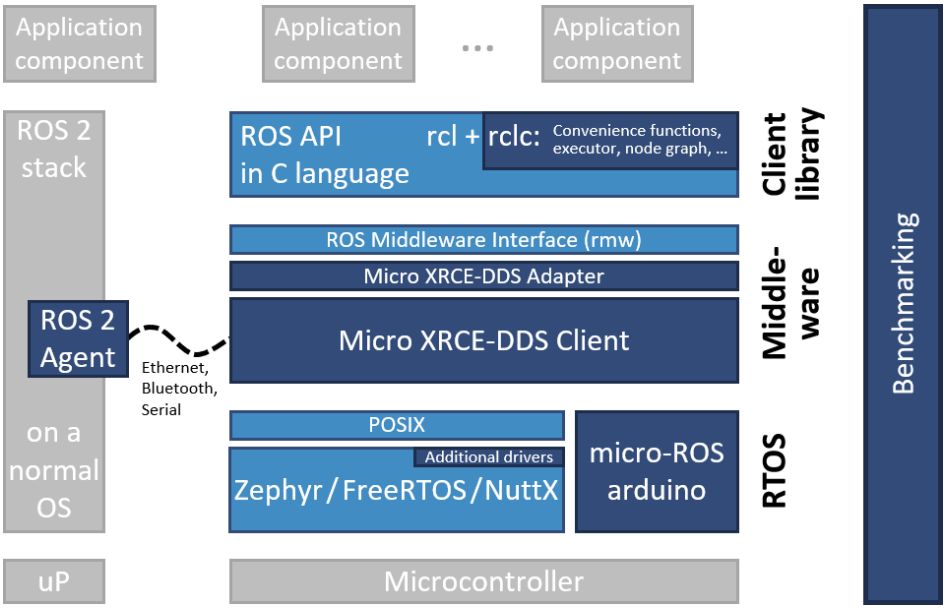
ROSAGENT IS NOW RUNNING.



MicroROS is now letting the teensy act as a node

Let's discuss how Micro_ros_agent communicates with micro_ros_arduino on teensy for a clearer understanding and what happens when i run

ros2 run micro_ros_agent micro_ros_agent serial —dev /dev/ttyACM0



from what i understand, when I run the code on teensy, the teensy firmware (currently mirco_ros_arduino.h) is running

we are connecting it to the computer which has ros2 and micro_ros_agent

the microcontroller RTOS and the ros2 agent will make use of the Clientlibrary (rcl) and use MicroXRCE-DDS client which acts like middleware

Micro XRCE-DDS | micro-ROS

- Micro XRCE-DDS Agent: a **C++11 out-of-the-box application** which implements the XRCE Agent functionality.
- Micro XRCE-DDS Client: a **C99 library** which implements the XRCE Client side functionality.

when you do ros2 run micro_ros_agent micro_ros_agent serial —dev /dev/ttyACM0

your agent becomes an internal node in ros2

it tells ros2 bro i am a node

it opens serial port dev/ttyACM0 and connects to the teensy by starting a XRCE server

and so they keep talking

im not 100% thorough with how this all works

but yes.

SO first, you run teensy build and upload

and then you run ros2 microros on wsl

`ros2 run micro_ros_agent micro_ros_agent serial –dev /dev/ttyACM0`

then check ros2 topic echo /gps/fix

Why did we choose String Message? Becase the data is in String format. like latitude and longitude values and all.

we are returnining as "Lat: -12.345678, Lon: 123.456789, Altitude: 1234.56 m"

so its best for string format

if we are returning in different way, then we can choose different message format

Why a fixed gpsPayload[96] buffer?m Predictable memory use: 96 bytes is enough for "Lat: -12.345678, Lon: 123.456789, Altitude: 1234.56 m"

Teensy code explanation

```
#include <Arduino.h>
#include <TinyGPS++.h>
#include <micro_ros_arduino.h>
#include <rcl/rcl.h>
#include <rclc/rclc.h>
#include <std_msgs/msg/string.h>
```

Here,

1. Simplified init functions for resource-constrained devices (micro_ros_arduino for the teensy)
2. Core ROS 2 C API: nodes, publishers, QoS, etc. (rcl.h)
3. Simplified init functions for resource-constrained devices (rclc/rclc.h)
4. std_msgs/msg/string.h

```
rcl_allocator_t allocator;           // Memory allocator for ROS 2 (uses malloc/free by de
fault)
rclc_support_t support;              // Initialization support struct (holds context)
rcl_node_t node;                     // ROS 2 node: "teensy_gps_node"
rcl_publisher_t gpsPublisher;        // Publisher to /gps/fix topic

std_msgs__msg__String gpsMsg;        // C struct for ROS 2 String message
char gpsPayload[96];                 // Fixed buffer for message content (avoids dynamic a
llocation)
```

Why did we choose String Message? Becase the data is in String format. like latitude and longitude values and all.

we are returnining as "Lat: -12.345678, Lon: 123.456789, Altitude: 1234.56 m"

so its best for string format

if we are returning in different way, then we can choose different message format

Why a fixed gpsPayload[96] buffer?m Predictable memory use: 96 bytes is enough for "Lat: -12.345678, Lon: 123.456789, Altitude: 1234.56 m"

`TinyGPSPlus gps;`

`HardwareSerial &gpsUart = Serial1;`

`constexpr uint32_t USB_SERIAL_BAUD = 115200;`
`constexpr uint32_t GPS_BAUD = 9600;`

This is error handling macro

```
void errorLoop() {
    while (true) {
        delay(100);
    }
}

#define RCCHECK(fn) do { if ((fn) != RCL_RET_OK) { errorLoop(); } } while (0)
#define RCSOFTCHECK(fn) do { (void)(fn); } while (0)
```

idk allat no mood

```
void setup() {
    Serial.begin(USB_SERIAL_BAUD);
  // Serial2.begin(9600, SERIAL_8N1, 16, 17); // (Baud, Config, RX_Pin, TX_Pin)
    gpsUart.begin(GPS_BAUD);

    set_microros_transports();
    delay(2000);

    allocator = rcl_get_default_allocator();

    RCCHECK(rclc_support_init(&support, 0, NULL, &allocator));
    RCCHECK(rclc_node_init_default(&node, "teensy_gps_node", "", &support));
    RCCHECK(rclc_publisher_init_default(
        &gpsPublisher,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
        "/gps/fix"
    ));

    gpsMsg.data.data = gpsPayload;
    gpsMsg.data.capacity = sizeof(gpsPayload);
    gpsMsg.data.size = 0;
}
```

This function `set_microros_transports();` (defined in `micro_ros_arduino.h`) tells micro-ROS **how to send/receive data**:

initialization order:

1. Hardware init: USB serial and GPS UART
2. Microros Transport setup: configure the transportation unit and wait for agent connection
3. ROS2 Core initialization load memory allocator, support, load the node and publisher creation

```
void loop() {
    while (gpsUart.available() > 0) {
        const char nmeaByte = static_cast<char>(gpsUart.read());
        gps.encode(nmeaByte);
    }
```

```
    if (gps.location.isUpdated() && gps.location.isValid() && gps.altitude.isValid()) {
        const int written = snprintf(
            gpsPayload,
            sizeof(gpsPayload),
            "Lat: %.6f, Lon: %.6f, Altitude: %.2f m",
            gps.location.lat(),
            gps.location.lng(),
            gps.altitude.meters()
        );

        if (written > 0) {
            gpsMsg.data.size = static_cast<size_t>(
                written < static_cast<int>(sizeof(gpsPayload))
                ? written
                : static_cast<int>(sizeof(gpsPayload)) - 1
            );
            RCSOFTCHECK(rcl_publish(&gpsPublisher, &gpsMsg, NULL));
        }
    }
}
```

PLATFORMIO.INI FILE:

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:teensy41]
platform = teensy
board = teensy41
framework = arduino
lib_ldf_mode = chain+
lib_archive = false
build_flags =
    -L"${PROJECT_LIBDEPS_DIR}/${PIOENV}/micro_ros_arduino/src/imxrt1062/fpv5-d16-hard"
    -Wl,--whole-archive
    -lmicroros
    -Wl,--no-whole-archive
lib_deps =
    mikalhart/TinyGPSPlus@^1.1.0
    mirs240x/micro_ros_arduino@^2.0.7-humble
```

from what i understand, -L"${PROJECT_LIBDEPS_DIR}/${PIOENV}/micro_ros_arduino/src/imxrt1062/fpv5-d16-hard"

is for linking a floating point ABI (application binary interface) for a floating point unit

an FPU is a unit which is used to process floating point numbers

bro ukw i do not want to go into the details

let the work be the work it is

ChatGPT chat link: https://chatgpt.com/share/69cf04e4-0f4c-8322-be57-ec1cc53aa6b7